

DEVOIR DE CLASSIFICATION

Bases de Machine Learning

Réalisé par : Najlaa Nacif

Master : BDCC

Année universitaire : 2025 – 2026

Exercice 1 : Classification avec TensorFlow

1. Préparation des données

Le jeu de données utilisé est **diabetes.csv**. La variable cible est **Outcome**, qui prend les valeurs suivantes :

- Outcome = 0 : patient non diabétique ;
- Outcome = 1 : patient diabétique.

Le dataset contient huit variables explicatives décrivant différentes caractéristiques médicales des patients.

Les données ont été divisées en deux ensembles :

- 80 % des observations sont utilisées pour l'apprentissage du modèle ;
- 20 % des observations sont réservées à l'évaluation.

Cette séparation permet d'évaluer les performances du modèle sur des données qui n'ont pas été utilisées pendant la phase d'entraînement.

2. Réseau de neurones TensorFlow

2-a Architecture proposée

Le réseau de neurones utilisé possède une architecture simple composée de :

- huit neurones d'entrée correspondant aux variables explicatives du dataset ;
- aucune couche cachée ;

- un neurone de sortie ;
- une fonction d'activation sigmoïde (*sigmoid*).

Cette architecture est équivalente à un modèle de régression logistique implémenté avec TensorFlow.

2-b Construction du modèle

Le modèle est construit à l'aide de la classe `Sequential` de TensorFlow.

Une couche `Dense` contenant un unique neurone de sortie avec une fonction d'activation **sigmoid** est ajoutée au réseau.

2-c Compilation du modèle

Le modèle est compilé avec :

- l'optimiseur **Adam** ;
- la fonction de perte **Binary Crossentropy** ;
- les métriques **Accuracy** et **Precision**.

L'optimiseur Adam est choisi pour sa rapidité de convergence et sa stabilité lors de l'apprentissage.

2-d Entraînement du modèle

Le réseau de neurones est entraîné pendant **100 epochs**.

Durant cette phase, TensorFlow ajuste automatiquement les poids et le biais afin de minimiser la fonction de coût.

2-e Rôle de la fonction `fit()`

La fonction `fit()` permet d'entraîner le réseau de neurones.

À chaque époque, une propagation avant est réalisée afin de calculer les prédictions. L'erreur entre les valeurs réelles et prédites est ensuite évaluée puis utilisée pour mettre à jour les paramètres du modèle grâce à l'optimiseur Adam.

2-f Nombre de paramètres

Le réseau possède :

- 8 poids associés aux variables d'entrée ;
- 1 biais.

Le nombre total de paramètres entraînables est donc :

```
[  
8+1=9  
]
```

Cette valeur est confirmée par la fonction `model.summary()`.

2-g Vérification des prédictions

Les poids et le biais appris par TensorFlow ont été récupérés à partir du modèle.

Les probabilités ont ensuite été recalculées manuellement à l'aide des expressions suivantes :

```
[  
z=w_1x_1+w_2x_2+\cdots+w_8x_8+b  
]
```

```
[  
\hat{y}=\sigma(z)=\frac{1}{1+e^{-z}}  
]
```

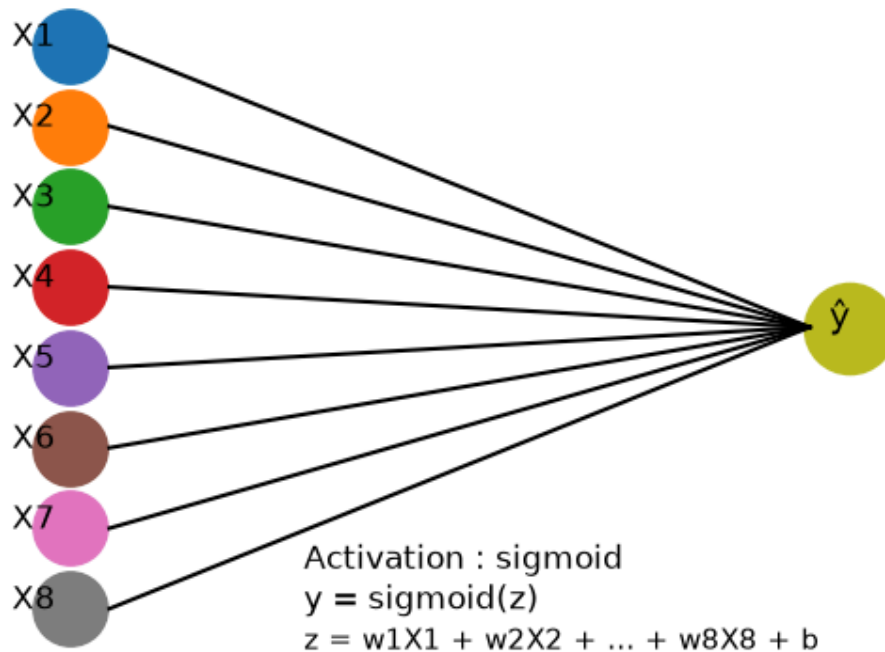
Les résultats obtenus sont identiques à ceux fournis par la méthode `predict()`, ce qui confirme le bon fonctionnement du modèle.

2-h Schéma du réseau

Le réseau de neurones comporte huit variables d'entrée directement reliées à un neurone de sortie.

Le neurone calcule une combinaison linéaire des variables d'entrée puis applique une fonction d'activation sigmoïde afin de produire une probabilité comprise entre 0 et 1.

Schéma du réseau de neurones



2-i Régularisation

Les techniques de régularisation permettent de réduire le risque de surapprentissage et d'améliorer la capacité de généralisation d'un modèle.

Parmi les méthodes les plus utilisées figurent :

- la régularisation L1 ;
- la régularisation L2 ;
- le Dropout ;
- l'Early Stopping.

Dans cette étude, le modèle comporte seulement neuf paramètres entraînaibles et ne présente pas de signe de surapprentissage. L'utilisation d'une méthode de régularisation n'est donc pas nécessaire.

3. Évaluation du modèle TensorFlow

Les performances obtenues sur l'ensemble de test sont les suivantes :

- Accuracy : **0.7143**
- Recall : **0.3774**
- F1-score : **0.4762**

La matrice de confusion obtenue est :

```
[
\begin{bmatrix}
90 & 11 \ \\
33 & 20
\end{bmatrix}
]
```

Le rapport de classification montre que le modèle obtient de meilleures performances pour la classe 0 que pour la classe 1.

Exercice 2 : Classification implémentée sans TensorFlow

Une régression logistique a été implémentée à partir de zéro en utilisant uniquement la bibliothèque **NumPy**.

Les poids et le biais sont initialisés à zéro puis ajustés progressivement grâce à l'algorithme de descente de gradient.

Après l'entraînement, les performances obtenues sur l'ensemble de test sont :

- Accuracy : **0.6688**
- Recall : **0.2264**
- F1-score : **0.3200**

La matrice de confusion obtenue est :

```
[
\begin{bmatrix}
91 & 10 \ \\
41 & 12
\end{bmatrix}
]
```

Comparaison des modèles

Modèle	Accuracy	Recall	F1-score
--------	----------	--------	----------

TensorFlow	0.7143	0.3774	0.4762
From Scratch	0.6688	0.2264	0.3200

Conclusion

Les résultats expérimentaux montrent que le modèle développé avec TensorFlow présente de meilleures performances que l'implémentation réalisée manuellement avec NumPy.

En particulier, il obtient une meilleure accuracy, un meilleur recall ainsi qu'un meilleur F1-score.

L'implémentation *From Scratch* reste néanmoins intéressante d'un point de vue pédagogique, car elle permet de mieux comprendre les mécanismes fondamentaux de la régression logistique, notamment la fonction sigmoïde, la descente de gradient et la mise à jour des paramètres du modèle.

Exercice 1 : Régression avec TensorFlow

1. Préparation des données

1-a Génération du dataset synthétique

Afin d'étudier un problème de régression, un jeu de données synthétique a été généré à l'aide de la bibliothèque **NumPy**.

La variable cible est construite selon la relation suivante :

$$\begin{aligned} &[\\ y &= 2X_1 + 3X_2 + 5 + \text{bruit} \\ &] \end{aligned}$$

Le dataset contient **100 observations**. Les variables explicatives (X_1) et (X_2) sont générées aléatoirement tandis qu'un bruit gaussien est ajouté afin de rendre le problème plus réaliste.

1-b Construction de X et y

Les deux variables d'entrée (X_1) et (X_2) sont regroupées dans une matrice de caractéristiques (X).

La variable cible (y) est ensuite calculée à partir de l'équation précédente.

1-c Division du dataset

Le jeu de données est séparé en deux ensembles :

- 80 % des observations sont utilisées pour l'entraînement du modèle ;
- 20 % des observations sont réservées à l'évaluation.

Cette séparation est effectuée à l'aide de la fonction `train_test_split`.

1-d Importance du découpage Training/Test

Le **Training Set** permet au modèle d'apprendre les relations entre les variables d'entrée et la variable cible.

Le **Test Set** est utilisé pour évaluer les performances du modèle sur des données jamais vues pendant l'apprentissage.

Cette méthode permet d'estimer la capacité de généralisation du modèle et d'éviter le phénomène de surapprentissage.

2. Réseau de neurones TensorFlow

2-a Architecture proposée

Le réseau de neurones utilisé possède une architecture simple composée de :

- deux neurones d'entrée ;
- aucune couche cachée ;
- un neurone de sortie ;
- une fonction d'activation **linéaire**.

Cette architecture est équivalente à un modèle de régression linéaire implémenté avec TensorFlow.

2-b Construction du modèle

Le modèle est construit à l'aide de la classe `Sequential` de TensorFlow.

Une couche `Dense` contenant un unique neurone de sortie est ajoutée au réseau.

2-c Compilation du modèle

Le modèle est compilé avec :

- l'optimiseur **Adam** ;
- la fonction de perte **Mean Squared Error (MSE)** ;
- la métrique **MSE**.

L'optimiseur Adam est choisi pour sa rapidité de convergence et sa stabilité lors de l'apprentissage.

2-d Entraînement du modèle

Le réseau de neurones est entraîné pendant **4000 epochs**.

Durant cette phase, TensorFlow ajuste automatiquement les poids et le biais afin de minimiser l'erreur quadratique moyenne.

2-e Rôle de la fonction `fit()`

La fonction `fit()` permet d'entraîner le réseau de neurones.

À chaque époque, une propagation avant est réalisée afin de calculer les prédictions. L'erreur entre les valeurs réelles et prédites est ensuite évaluée puis utilisée pour mettre à jour les paramètres du modèle grâce à l'optimiseur Adam.

2-f Nombre de paramètres

Le réseau possède :

- 2 poids associés aux variables d'entrée ;
- 1 biais.

Le nombre total de paramètres entraînaibles est donc :

```
[  
2+1=3  
]
```

Cette valeur est confirmée par la fonction `model.summary()`.

2-g Poids et biais appris

Après l'entraînement du modèle, les paramètres obtenus sont :

- ($w_1 = 2.0225$)
- ($w_2 = 3.0893$)
- ($b = 4.3442$)

Ces valeurs sont très proches des paramètres théoriques utilisés lors de la génération du dataset.

2-h Schéma du réseau

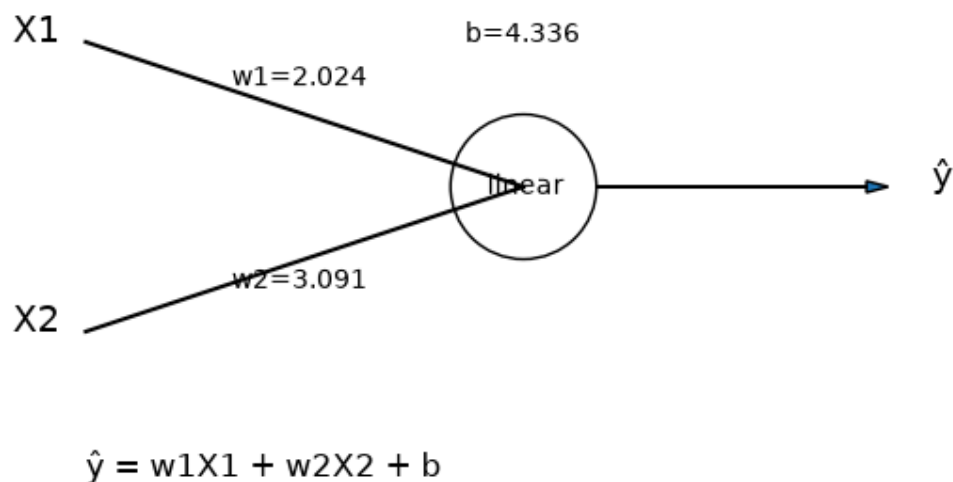
Le réseau de neurones comporte deux variables d'entrée directement reliées à un neurone de sortie.

Le neurone effectue une combinaison linéaire des variables d'entrée selon l'expression :

$$\hat{y} = w_1X_1 + w_2X_2 + b$$

La fonction d'activation utilisée est une fonction **linéaire**.

2-h : Schéma du réseau de neurones



2-i Régularisation

Les techniques de régularisation permettent de réduire le risque de surapprentissage et d'améliorer la capacité de généralisation d'un modèle.

Parmi les méthodes les plus utilisées figurent :

- la régularisation L1 ;
- la régularisation L2 ;
- le Dropout ;
- l'Early Stopping.

Dans cette étude, le modèle comporte seulement trois paramètres entraînables et ne présente pas de signe de surapprentissage. L'utilisation d'une méthode de régularisation n'est donc pas nécessaire.

3. Prédiction

3-a Utilisation de la fonction `predict()`

La fonction `predict()` permet de calculer les valeurs prédites pour de nouvelles observations.

Les prédictions sont obtenues à partir des données du test set.

3-b Prédiction matricielle

Les prédictions ont également été calculées manuellement à l'aide de l'expression :

```
[  
  \hat{y} = XW+b  
]
```

Les résultats obtenus sont identiques à ceux fournis par la fonction `predict()`, ce qui confirme le bon fonctionnement du modèle.

4. Évaluation du modèle TensorFlow

4-a Évaluation sur le Training Set

Les performances obtenues sur l'ensemble d'entraînement sont :

- MSE = **3.9964**
- ($R^2 = 0.9514$)

Le graphique des résidus montre que les erreurs sont réparties de manière aléatoire autour de zéro, ce qui indique un bon ajustement du modèle aux données d'apprentissage.

4-b Évaluation sur le Test Set

Les performances obtenues sur l'ensemble de test sont :

- MSE = **3.5859**
- ($R^2 = 0.9682$)

Ces résultats montrent que le modèle généralise correctement sur des données non utilisées durant l'entraînement.

4-c Utilité de l'évaluation sur Train et Test

L'évaluation sur le training set permet de mesurer la qualité de l'apprentissage.

L'évaluation sur le test set permet de vérifier la capacité du modèle à prédire correctement de nouvelles observations.

La comparaison entre ces deux ensembles permet également de détecter un éventuel phénomène de surapprentissage.

Exercice 2 : Régression implémentée sans TensorFlow

Une régression linéaire a été implémentée à partir de zéro en utilisant uniquement la bibliothèque **NumPy**.

Les poids et le biais sont initialisés à zéro puis ajustés progressivement grâce à l'algorithme de descente de gradient.

Après l'entraînement, les paramètres obtenus sont :

- ($w_1 = 1.9879$)
- ($w_2 = 3.0471$)
- ($b = 4.7620$)

Les performances obtenues sur l'ensemble de test sont :

- $MSE = 3.5422$
- ($R^2 = 0.9686$)

Les prédictions obtenues avec l'implémentation manuelle sont très proches de celles produites par TensorFlow.

Comparaison des modèles

Méthode	MSE	R^2
---------	-----	-------

TensorFlow 3.5859 0.9682

From Scratch 3.5422 0.9686

Conclusion

Les résultats expérimentaux montrent que les deux approches produisent des performances très similaires.

L'implémentation TensorFlow facilite l'entraînement grâce à l'optimiseur Adam et permet une convergence rapide du modèle.

L'implémentation *From Scratch* reste néanmoins très intéressante d'un point de vue pédagogique, car elle permet de mieux comprendre les mécanismes fondamentaux de la descente de gradient, du calcul des gradients ainsi que de la mise à jour des paramètres du modèle.